WikiApp

Viva Preparation Guide

From zero to confident in one document

BIT235 - Object Oriented Programming
Assessment 2 - Parts A & B
Semester 1, 2026

Author: Movindu Lochana
Student ID: s1577380

This guide assumes you have never used Spring Boot before. Every term is explained the first time it appears, and every line of code is broken down in plain English. Read it once cover-to-cover, then re-read the parts that feel shaky.

How to use this guide

This is a separate, more beginner-friendly companion to the technical **WikiApp_Code_Walkthrough.pdf**. It is written assuming you have NEVER used Spring Boot before. Every concept is explained before any code is shown.

Suggested study plan

Day before viva (about 2 hours total):

• **30 min** - Read Part 1 (Background & Concepts) carefully. Stop and re-read anything that does not make sense.

• **40 min** - Read Part 2 (File-by-file walkthrough). Open each file in your editor side-by-side with this PDF.

• **20 min** - Read Part 3 (the demo script) and physically click through the app while saying the explanation out loud.

• **30 min** - Read Part 4 (likely questions) and try to answer each one out loud BEFORE looking at the suggested answer.

Day of viva (15 min):

• Re-read the One-Minute Pitch section.

• Re-read the Demo Script.

• Glance at the Cheat Sheet for any terms you forgot.

KEY IDEA: If the marker asks something you genuinely do not know, say *'I am not 100% sure, but my best guess is...'* and reason out loud. Markers reward thoughtful uncertainty more than confident wrong answers.

What this guide covers

Part 1 - Background & Concepts	Explains web apps, HTTP, MVC, Spring Boot, JPA, sessions - everything you need to understand the code BEFORE you read it.
Part 2 - File Walkthrough	Every file in the project, line by line, in plain English.
Part 3 - The Demo Script	Exactly what to click in what order during the viva, with what to say.
Part 4 - Viva Q&A	30+ likely questions with model answers and reasoning.
Part 5 - Quick Reference Cheat Sheet	One-page summary you can glance at right before the viva.

PART 1

Background & Concepts

1.1 What is a web application?

A web application is a program that runs on a server and answers requests from web browsers. When you type a URL into your browser, the browser sends a message over the internet to a server, the server prepares a response (usually an HTML page), and your browser shows it.

The two parties involved

• **Browser (the client):** Chrome, Firefox, Edge, etc. It sends requests and displays the response.

• **Server (the application):** A program running somewhere - in our case, on your own laptop on port 8080. It receives requests and sends back responses.

> **KEY IDEA:** In your project, the **browser** is Chrome/Brave on your laptop, and the **server** is your Spring Boot app, also running on your laptop. They talk to each other through the URL **localhost:8080**.

What is HTTP?

HTTP (HyperText Transfer Protocol) is the language browsers and servers use to talk. Every conversation has the same structure:

1. Browser sends a request, which has:

- A **method** - usually GET (just reading) or POST (sending data)
- A **URL** - what page is being requested, e.g. /login
- Optionally, a **body** - data being sent, e.g. form fields

2. Server sends a response, which has:

- A **status code** - 200 = OK, 302 = redirect, 404 = not found, 500 = server crashed
- A **body** - usually HTML the browser will display

Concrete example from your project

```
1. You type http://localhost:8080/wiki and press Enter.

2. Browser sends:
       GET /wiki HTTP/1.1
       Host: localhost:8080

3. Your Spring Boot app receives it. WikiController.listArticles() runs.
   It asks the database for all articles, then renders wiki/list.html.

4. Server sends back:
       HTTP/1.1 200 OK
       Content-Type: text/html
       <html>...the rendered page...</html>

5. Browser displays the HTML.
```

1.2 What is Spring Boot?

Spring is a huge collection of Java libraries that help you build server-side applications. **Spring Boot** is a wrapper around Spring that makes it easy to start: it bundles sensible defaults so you can have a running web app in about 10 lines of Java.

What does Spring Boot give us, specifically?

An embedded web server (Tomcat)	We do not have to install or configure a web server separately. When we run our app, Tomcat starts inside our app and listens on port 8080.
Annotations like @Controller, @Service	Plain Java classes become web-aware just by adding these labels. No XML configuration needed.
Automatic dependency injection	Spring creates objects we need (like our AuthService) and passes them where they belong, so we never write 'new AuthService()' ourselves.
Auto-configured database support	Add the JPA dependency and tell it where the database is - Spring Boot does the rest.
Thymeleaf integration	Spring Boot auto-discovers HTML files in src/main/resources/templates/ and lets us use ${} expressions inside them.

KEY IDEA: Spring Boot does not magically write your code. What it does is take care of all the boring setup so you can focus on the few classes that matter: your Controllers, Services and Repositories.

The trade-off

Spring Boot annotations look magical because there is no obvious code behind them. **@Controller** on a class makes it a controller. **@Autowired** on a constructor makes Spring fill in the parameter. This is great for productivity but it means YOU need to know what each annotation actually does - because the marker will probably ask.

This guide explains every annotation you used. The Cheat Sheet at the end is a single-page summary you can glance at before the viva.

1.3 What is MVC?

MVC stands for **Model - View - Controller**. It is a way of organising code so each piece has one job. The marker WILL ask about MVC - it is one of the rubric's named criteria.

Restaurant analogy

Imagine ordering food at a restaurant:

• **You** - the customer (the browser, the user)

• **The waiter** - takes your order, brings the food (the Controller)

• **The chef** - prepares the food using ingredients (the Service in our app)

• **The pantry** - stores the ingredients (the Repository / database)

• **The menu and the plate** - what you see and read (the View)

• **The order ticket / dish** - the data being passed around (the Model)

In code

Model	The data. In our project: **LoginForm.java**, **Article.java**, **Admin.java**. Plus **Model** from Spring, which is a basket we put data into for the view to read.
View	What the user sees. In our project: every **.html** file in templates/. Thymeleaf processes them to fill in the data.
Controller	Decides what to do when a request arrives. In our project: **AuthController**, **WikiController**, **AdminController**. They route requests to the right Service and pick which View to render.

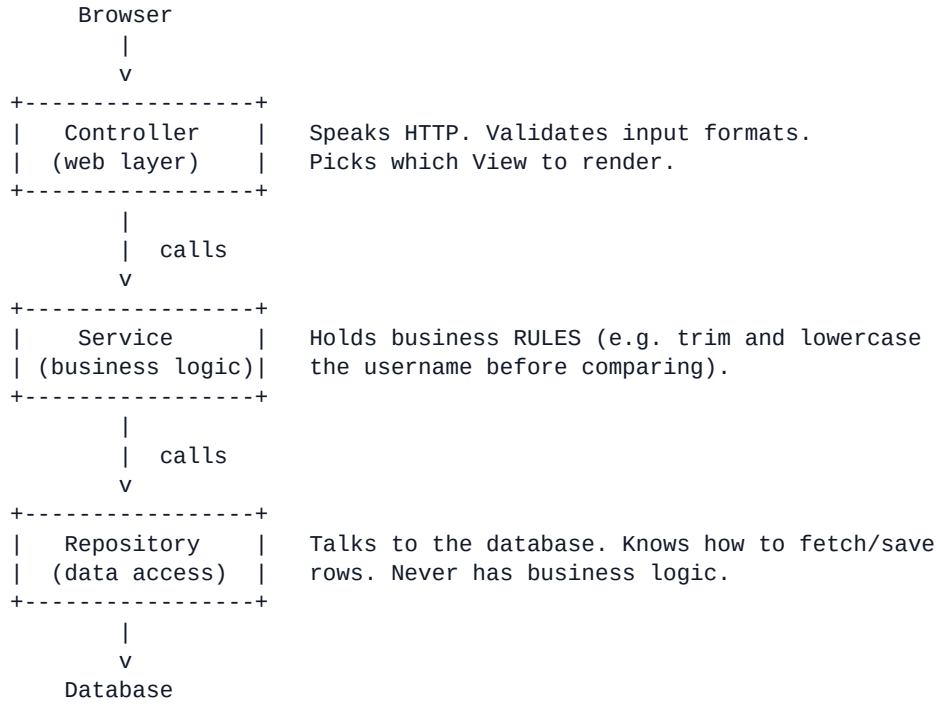
KEY IDEA: The Controller is a TRAFFIC COP. It does not contain business logic and it does not store data. Its only job is: receive request -> ask Service -> pick View. If a Controller method has 50 lines of logic in it, something is wrong.

Why bother?

Because if you mix everything together, changing one thing breaks ten others. By keeping each layer focused, you can swap one out without rewriting the rest. The clearest example: in Part A we used hard-coded credentials in **UserRepository**. In Part B we replaced that with a real database (**AdminRepository**) - and the Controller and Service code did not need to change. That is the point.

1.4 The three-layer architecture

Inside the Controller, our project follows a strict layering pattern. The marker calls this 'layered design' in the rubric.



Each layer has ONE responsibility:

Controller	Receive HTTP requests, hand off to Service, choose View. NO business logic, NO database calls. Just routing.
Service	All the business rules live here. Validation, time-stamping, decisions about what is allowed. NO HTTP, NO database SQL.
Repository	Database queries only. NO business rules. Returns plain Java objects.

KEY IDEA: If you remember nothing else: **Controller routes, Service decides, Repository fetches**. That is the whole architecture in one sentence.

Concrete example: the login flow

```
1. Browser POSTs to /login with username and password.

2. AuthController.processLogin() runs.
   - It does NOT compare passwords. Not its job.
   - It calls authService.isAuthenticated(loginForm).

3. AuthService.isAuthenticated() runs.
   - Trims spaces. Lower-cases the username.
   - Calls adminRepository.findByUsername(cleanedUsername).
   - It does NOT write any SQL. Not its job.

4. AdminRepository.findByUsername() runs.
   - It is just an interface; Spring wrote the SQL automatically.
   - Returns Optional<Admin> from the database.
```

```
5. AuthService gets the Admin back, compares passwords with .equals().
   Returns true or false.

6. AuthController gets true/false back.
   - On true: stores username in HttpSession, returns "redirect:/admin".
   - On false: returns "error".

7. Spring renders the chosen view and sends the HTML to the browser.
```

HOW TO SAY IT: “The Controller routes the request, the Service decides if the login is valid, and the Repository looks up the admin in the database. Each layer has one job. That is why my code is easy to maintain.”

1.5 What is JPA? What is a database?

A **database** is a program that stores data and answers queries about it. The most common type is a **relational database**, which stores data in tables - like spreadsheets - with rows and columns.

Our project uses **H2**, a small relational database that runs inside our app. It stores its data in a file called **./data/wikidb.mv.db**. When the app stops, the data stays in that file; when the app starts again, the data is still there.

What is a table?

A table holds rows of related data. Our project has two tables:

```
ARTICLE table:
  id (BIGINT)   |  title (VARCHAR)        |  category    |  content      |  last_updated
  --------------+-------------------------+--------------+---------------+-----------------
  1             |  Welcome to WikiApp     |  General     |  This is...   |  2026-05-03 10:00
  2             |  What is MVC?           |  Programming |  MVC stands.. |  2026-05-03 10:00
  ...

ADMIN table:
  id  |  username  |  password
  ----+------------+----------
  1   |  movindu   |  123
```

What is JPA?

JPA (Java Persistence API) is a standard way to map Java objects to database tables. Instead of writing SQL like 'SELECT * FROM article WHERE id = 3', you call **articleRepository.findById(3)** and get an Article object back. JPA generates the SQL for you.

Hibernate is the most popular JPA implementation, and Spring Boot ships with it by default. So when you read 'JPA' in our code, Hibernate is the actual library doing the work.

How JPA knows what tables to create

When the app starts, JPA scans for classes marked **@Entity**. For each one, it makes sure a table with the same name exists, with columns matching the class's fields. Our setting **spring.jpa.hibernate.ddl-auto=update** tells JPA to create missing tables and add new columns automatically.

> **KEY IDEA:** @Entity classes mirror tables. Each @Entity object is one row. The fields are the columns. @Id marks which field is the primary key (the unique identifier for each row).

What is JpaRepository?

It is a Spring interface that provides standard database operations: **findAll, findById, save, deleteById** and many more. By extending it (**extends JpaRepository<Article, Long>**), our interfaces inherit all those methods - and Spring writes the implementation for us at startup. We literally do not write any SQL.

Custom queries from method names

Spring Data goes further: if you declare a method following its naming convention, Spring will write the SQL based on the name alone:

```
// In ArticleRepository.java:
List<Article> findByCategoryIgnoreCase(String category);
```

```
// Spring reads the method name and generates:
//   SELECT * FROM article WHERE LOWER(category) = LOWER(?)
// You never see this SQL. It just works.
```

1.6 What is HttpSession?

HTTP is **stateless** - each request is independent. The server has no way to know that 'this request' came from the same browser as 'last request'. So how do logged-in pages stay logged in?

Cookies and session ids

When a browser visits the site for the first time, Tomcat assigns it a long random number called a **session id** and sends it back as a cookie. The browser stores this cookie and sends it along with every future request. Tomcat uses the cookie to identify the same browser across requests.

On the server side, Tomcat keeps a **session object** for each browser with that session id - a small in-memory storage area where we can put values. That session object is what we get when we ask for **HttpSession** in a controller method.

How we use it

```
// In AuthController.processLogin() after successful login:
session.setAttribute("loggedInAdmin", "movindu");
//                   ^ key             ^ value

// Later, in AdminController.dashboard():
Object value = session.getAttribute("loggedInAdmin");
if (value == null) {
    // Not logged in - send to login page
    return "redirect:/login";
}

// In AuthController.logout():
session.invalidate();   // throws away EVERY value stored in this session
```

KEY IDEA: Sessions are how we know who is logged in. setAttribute stores the username; getAttribute reads it; invalidate() forgets everything. The browser cookie keeps the same session alive across page loads.

1.7 What is Thymeleaf?

Thymeleaf is a **template engine**. We write HTML files with special **th:*** attributes that are placeholders for data. When the page is requested, Thymeleaf reads the model and replaces those placeholders with actual values before sending the HTML to the browser.

Example

```
<!-- The template (welcome.html): -->
<p>Hello <strong th:text="${username}">user</strong>, welcome back.</p>

<!-- After Thymeleaf processes it, the browser sees: -->
<p>Hello <strong>movindu</strong>, welcome back.</p>
```

Why is the word 'user' there?

That is the **fallback content**. If you open the .html file directly in a browser without Thymeleaf running, you still see something readable. When Spring serves the page, Thymeleaf overrides 'user' with the actual value of **${username}**.

The most useful Thymeleaf attributes

th:text="${x}"	Replace this tag's text with the value of x. HTML-escapes automatically.
th:href="@{/login}"	Build a URL for an <a> tag's href.
th:action="@{/login}"	Build a URL for a form's submit target.
th:object="${form}"	Bind the whole form to this Java object from the model.
th:field="*{username}"	Bind one input to a property of the th:object. Sets name, id and value automatically.
th:each="a : ${list}"	For-each loop. Render this element once per item.
th:if="${cond}"	Render this element only when cond is true.
th:errors="*{name}"	Show validation errors for this field.
@{/path}	URL expression - safer than hard-coding paths.
${variable}	Variable expression - reads from the Model.
***{property}**	Selection expression - reads from the th:object.

WATCH OUT: The dollar sign expression **${username}** reads from the Model. The asterisk expression ***{username}** reads from the object the form is bound to via th:object. They look similar but they read from different places.

1.8 The five most important annotations

If the marker points at any annotation in your code and asks 'what does this do?', you need to be able to answer. These five appear most often.

@Controller

Marks a class as a web controller. Spring scans for it at startup, creates one instance, and routes incoming HTTP requests to its methods based on their @GetMapping / @PostMapping URLs.

HOW TO SAY IT: “@Controller tells Spring this is a web controller and methods return view names rather than data.”

@Service

Marks a class as a business-logic bean. Functionally identical to @Component, but the name signals intent: business rules go here.

HOW TO SAY IT: “@Service marks a class that holds business logic. Spring manages a single instance of it that any controller can use.”

@Repository

Marks a class or interface as data access. Functionally identical to @Component, but the name signals intent: database stuff goes here.

@Autowired

Tells Spring to inject the required dependency. We use it on the constructor: when Spring builds our class, it looks at the constructor's parameters, finds matching beans, and passes them in.

```
@Autowired
public AuthService(AdminRepository adminRepository) {
    this.adminRepository = adminRepository;
}
// At startup Spring sees: "AuthService needs an AdminRepository to be built.
// I have an AdminRepository bean - here you go."
```

@GetMapping / @PostMapping

Map a method to a URL. @GetMapping handles requests when the user just wants to view a page. @PostMapping handles form submissions where data is being sent.

```
@GetMapping("/login")           // when browser asks for /login
public String showLoginPage(...) { ... }

@PostMapping("/login")          // when browser SUBMITS a form to /login
public String processLogin(...) { ... }
```

PART 2

File-by-file walkthrough

Each section follows the same structure:

- **What is this file?** - one sentence
- **Why does it exist?** - the reason the file is there at all
- **The code** - shown in chunks with explanation
- **Likely viva question** - what the marker may ask, and how to answer

2.1 WikiAppApplication.java

What is this file? The starting point of the entire application.

Why does it exist? When you run a Java program, the JVM looks for a class with a **main** method and runs it. This is that class. It tells Spring Boot to start everything up.

```
package com.wikiapp;
```

Says this class lives in the com.wikiapp package. Packages are like folders that group related classes. The folder structure on disk must match: src/main/java/com/wikiapp/.

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Brings in the two Spring classes we are about to use. Without these imports the names below would be undefined.

```
@SpringBootApplication
public class WikiAppApplication {
```

@SpringBootApplication is the most important annotation in the project. It is a SHORTCUT for three things at once: (1) **@Configuration** - this class can define beans; (2) **@EnableAutoConfiguration** - Spring sets up sensible defaults like the embedded Tomcat web server, the Thymeleaf engine, and the H2 database connection; (3) **@ComponentScan** - Spring searches THIS package and ALL sub-packages for other annotated classes (@Controller, @Service, @Repository, @Entity) and registers them automatically.

```
    public static void main(String[] args) {
        SpringApplication.run(WikiAppApplication.class, args);
    }
}
```

Standard Java main method - the JVM runs this first. **SpringApplication.run(...)** hands control to Spring Boot. Behind the scenes Spring Boot starts the embedded Tomcat web server, scans for our annotated classes, creates one instance of each, wires them together with dependency injection, connects to the database, and starts listening on port 8080. From this single line, the whole app comes alive.

Likely viva question: “Walk me through what @SpringBootApplication actually does.”

HOW TO SAY IT: “It is a shortcut for three annotations: @Configuration so this class can define beans, @EnableAutoConfiguration so Spring sets up defaults like the Tomcat server and the database, and @ComponentScan so Spring finds all my @Controller, @Service, @Repository and @Entity classes in this package and below.”

2.2 LoginForm.java (Model)

What is this file? A simple Java object that holds the username and password the user types into the login form.

Why does it exist? When the form is submitted, Spring needs an object to copy the typed values into. This class is that object. It also declares validation rules with @NotBlank.

```
@NotBlank(message = "Username is required")
private String username;

@NotBlank(message = "Password is required")
private String password;
```

Two private fields holding the form values. **@NotBlank** says the field must not be null and must not be empty/whitespace - if validation runs and the field is blank, the supplied message becomes an error. **private** means this field can only be accessed inside this class - other classes must use the getter and setter. Hiding fields like this is called **encapsulation**, an OOP principle.

```
public LoginForm() {
}
```

A no-argument constructor. We do not write code inside it - it just exists. Spring needs this so it can build a blank LoginForm before calling setUsername() and setPassword() with the form values.

```
public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}
```

Getter and setter for the username field. The getter just returns the value. The setter writes a new value. **this.username** means 'the field of this object', distinguishing it from the parameter that has the same name.

Likely viva question: “What is a POJO?”

HOW TO SAY IT: “Plain Old Java Object - a simple class with private fields, a no-args constructor, and getters and setters. LoginForm is a POJO. POJOs are the standard way to carry data around in a Java application.”

2.3 AuthService.java (Service layer)

What is this file? The business-logic class that decides whether a login is valid.

Why does it exist? We do not want validation rules scattered across our controller. By keeping them here, the rules are in one place, easy to find, and reusable.

```
@Service
public class AuthService {
```

@Service tells Spring this is a business-logic bean. Spring creates one instance at startup and reuses it everywhere it is needed.

```
private final AdminRepository adminRepository;

@Autowired
public AuthService(AdminRepository adminRepository) {
    this.adminRepository = adminRepository;
}
```

Constructor dependency injection. Our class needs an AdminRepository to do its job. We do not create one with 'new' - we declare it as a constructor parameter, mark the constructor with @Autowired, and Spring passes the bean it created earlier into our constructor at startup. The **final** keyword means we cannot reassign this reference later, which is safer.

```
public boolean isAuthenticated(LoginForm form) {
    if (form == null) return false;

    String username = form.getUsername();
    String password = form.getPassword();
```

Defensive null check (form should not be null but it is good practice). Then we read the username and password from the form using the getters defined in LoginForm.

```
    if (username == null || username.trim().isEmpty()) return false;
    if (password == null || password.trim().isEmpty()) return false;
```

Reject empty or whitespace-only inputs. **.trim()** removes leading and trailing whitespace from a string. **.isEmpty()** returns true when the string has zero characters. The double-pipe || means OR - if EITHER condition is true, we return false.

```
    String cleanedUsername = username.trim().toLowerCase();
```

This is a business rule. By trimming and lower-casing, the user can type 'Movindu', 'movindu' or ' MOVINDU ' and they all match. Note: we do NOT lower-case the password. Passwords must be case-sensitive.

```
    Optional<Admin> maybeAdmin = adminRepository.findByUsername(cleanedUsername);
    if (maybeAdmin.isEmpty()) return false;
```

Ask the repository to find the admin by username. **Optional<Admin>** means 'either an Admin or nothing' - a wrapper that forces us to handle the empty case explicitly. If no admin with that username exists, we cannot log them in - return false.

```
    Admin admin = maybeAdmin.get();
    return admin.getPassword().equals(password);
```

```
}
```

Pull the Admin out of the Optional with .get(). Compare the stored password to what the user typed using **.equals()** - this checks the actual character contents. **NEVER use ==** to compare strings in Java - that compares memory addresses, not content.

Likely viva question: “Why did you put validation in the Service and not the Controller?”

HOW TO SAY IT: “Separation of concerns. The Controller's job is HTTP routing. Validation rules are business logic, so they live in the Service. This way the rules are reusable, the Controller stays small, and I can write unit tests for the validation without involving HTTP at all.”

2.4 AuthController.java (Controller layer)

What is this file? Receives HTTP requests for /, /login, /logout and decides which view to render.

Why does it exist? Every URL the user can visit needs a controller method. This class handles everything related to logging in and out.

```
@Controller
@RequestMapping("/")
public class AuthController {
```

@**Controller** marks this as a web controller - methods return view names. @**RequestMapping("/")** sets the URL prefix at class level. Since it is just '/', methods inside set their own full URL.

```
public static final String SESSION_ADMIN_KEY = "loggedInAdmin";
```

A constant for the session key. Using a constant prevents typos: if I write 'loggedInAdmin' in one place and 'loggedInAdminUser' in another, my logic breaks silently. With the constant, the IDE catches typos at compile time.

```
@GetMapping({"/", "/login"})
public String showLoginPage(Model model) {
    model.addAttribute("loginForm", new LoginForm());
    return "login";
}
```

Handles GET requests for either / or /login. We put an empty LoginForm into the **Model** (Spring's container for data going to the view) so the form has something to bind to via th:object. Returning 'login' tells Spring to render **templates/login.html**.

```
@PostMapping("/login")
public String processLogin(@ModelAttribute("loginForm") LoginForm loginForm,
                           HttpSession session,
                           Model model) {
    boolean ok = authService.isAuthenticated(loginForm);
```

Handles the POST when the form is submitted. @**ModelAttribute** tells Spring: 'build a LoginForm by calling its setters with the matching form fields, then pass it in here'. **HttpSession** is the session object for the current browser. We ask the Service if the credentials are valid.

```
    if (ok) {
        String cleaned = loginForm.getUsername().trim().toLowerCase();
        session.setAttribute(SESSION_ADMIN_KEY, cleaned);
        return "redirect:/admin";
    }
```

On success, store the cleaned username in the session under our constant key. Then return **'redirect:/admin'**. The 'redirect:' prefix tells Spring to send a 302 HTTP response with a Location header pointing to /admin, which makes the browser load /admin freshly. This avoids the 'resubmit form?' warning when the user refreshes the page.

```
    model.addAttribute("errorMessage",
            "Invalid username or password. Please try again.");
    return "error";
}
```

On failure, put a message into the Model and render error.html.

```
@GetMapping("/logout")
public String logout(HttpSession session) {
    session.invalidate();
    return "redirect:/login";
}
```

Logout - throws away every value stored in this session, so subsequent requests look unauthenticated. Then redirect back to the login page.

Likely viva question: “Why did you redirect after login instead of rendering the dashboard directly?”

HOW TO SAY IT: “Two reasons. First, the URL in the address bar changes from /login to /admin so the user can bookmark or refresh without resubmitting the form. Second, browsers warn 'do you want to resubmit the form?' when you refresh a POST result - the redirect avoids that warning. This is a well-known web pattern called Post-Redirect-Get.”

2.5 Article.java (Part B - Entity)

What is this file? A JPA entity that maps to the ARTICLE table in the database.

Why does it exist? We need a way to store articles. Each Article object is one row in the table; each field is a column. JPA handles all the SQL.

```
@Entity
public class Article {
```

@**Entity** tells JPA to create a database table for this class. The table name defaults to the class name (ARTICLE).

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

@**Id** marks the primary key - the unique identifier for each row. @**GeneratedValue(IDENTITY)** tells the database to auto-generate the id whenever a new row is inserted. So we never set the id ourselves - it comes back populated after save().

```
@NotBlank(message = "Title is required")
@Column(nullable = false)
private String title;
```

@**NotBlank** validates the form input. @**Column(nullable = false)** makes the database column NOT NULL too - so even if validation is somehow bypassed, the database itself rejects nulls.

```
@NotBlank(message = "Content is required")
@Column(nullable = false, length = 5000)
private String content;
```

Default VARCHAR length is 255 characters. **length = 5000** tells JPA to make this column big enough for long article content.

```
private LocalDateTime lastUpdated;
```

When the article was last saved. We set it in ArticleService.save() so the Service layer owns this rule. **LocalDateTime** is the modern Java date/time type.

```
public Article() { }

public Article(String title, String category, String content) {
    this.title = title;
    this.category = category;
    this.content = content;
    this.lastUpdated = LocalDateTime.now();
}
```

JPA requires the no-args constructor (so it can build a blank Article and fill it from the database). The convenience constructor lets us create articles in code more easily.

Likely viva question: “What is the difference between @NotBlank and @Column(nullable=false)?”

HOW TO SAY IT: “@NotBlank is a Java validation rule that runs when a form is submitted - if the value is null or empty, the form shows an error. @Column(nullable=false) is a database constraint - the database itself rejects null values when you try to save. Using both is defence in depth: even

if the validation is somehow bypassed, the database still says no.”

2.6 Admin.java (Part B - Entity)

What is this file? A JPA entity for administrator accounts. Replaces Part A's hard-coded credentials.

Why does it exist? Part A had username and password as Java constants. Part B moves them into the database so we could in principle have multiple admins, change passwords without re-deploying, etc.

```
@Entity
public class Admin {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;
```

Standard entity with three fields. **unique = true** on the username column is special: the database itself rejects any attempt to insert two rows with the same username, even if our Java code somehow allowed it.

Likely viva question: “Why is the password stored in plain text?”

> **HOW TO SAY IT:** “The brief did not require hashing and we wanted the code easy to explain. In a real-world application I would hash passwords using BCrypt or Argon2 before storing - that way if the database leaks, attackers cannot just read everyone's passwords.”

2.7 ArticleRepository.java (Part B - Data Access)

What is this file? An INTERFACE that gives us full database access without writing any SQL.

Why does it exist? JpaRepository provides standard CRUD methods for free. By extending it we get findAll, findById, save, deleteById and many more, with zero implementation code.

```
public interface ArticleRepository extends JpaRepository<Article, Long> {
```

This is the most powerful single line in the project. By saying our interface extends **JpaRepository<Article, Long>**, we declare that this is a repository for Article entities whose primary key is a Long. Spring automatically writes a class that implements every method - including the SQL - at startup. We never see that class; it just works.

```
    List<Article> findByCategoryIgnoreCase(String category);

    List<Article> findByTitleContainingIgnoreCase(String text);
}
```

Custom queries with NO implementation code. Spring Data parses these method names and writes the SQL automatically: **findBy** means SELECT, **Category** matches the field, **IgnoreCase** wraps both sides in LOWER(). The result is SELECT * FROM article WHERE LOWER(category) = LOWER(?). We just write the method signature; Spring writes the implementation.

Likely viva question: “If this is just an interface, who provides the actual code?”

HOW TO SAY IT: “Spring Data JPA. At application startup it scans for interfaces extending JpaRepository, then generates a class at runtime that implements every method - including the SQL for our custom finder methods. I never have to write any SQL or see the implementation; it is created for me.”

2.8 AdminRepository.java (Part B - Data Access)

What is this file? Same idea as ArticleRepository, but for the Admin entity.

Why does it exist? Replaces Part A's hard-coded UserRepository. The shape is identical to ArticleRepository, just for a different entity.

```
public interface AdminRepository extends JpaRepository<Admin, Long> {

    Optional<Admin> findByUsername(String username);
}
```

Spring builds the SQL **SELECT * FROM admin WHERE username = ?** from the method name. The return type is **Optional<Admin>** - either an Admin or nothing. This forces the caller to handle the missing case explicitly: instead of getting a null and crashing, you have to call .isEmpty() or .get() first. This is a small but important safety improvement.

Likely viva question: “Why Optional instead of just Admin?”

HOW TO SAY IT: “Because the lookup might find nothing. If the return type were just Admin, the method would have to return null when no admin matched - and the caller could forget to check, leading to a NullPointerException. Optional is a wrapper that makes the missing case impossible to ignore: the caller has to handle it explicitly.”

2.9 ArticleService.java (Part B - Business Logic)

What is this file? Business logic for managing articles. Sits between AdminController/WikiController and ArticleRepository.

Why does it exist? We do not want our Controllers calling the Repository directly. The Service holds business rules (like time-stamping on save) and provides a clean API for the controllers.

```
@Service
public class ArticleService {

    private final ArticleRepository articleRepository;

    @Autowired
    public ArticleService(ArticleRepository articleRepository) {
        this.articleRepository = articleRepository;
    }
```

Same pattern as AuthService. Constructor injection brings in the repository.

```
public List<Article> findAll() {
    return articleRepository.findAll();
}

public Optional<Article> findById(Long id) {
    return articleRepository.findById(id);
}
```

Thin wrappers. The Service does not add anything here - we could have had the controllers call the repository directly. But keeping the Service in the middle gives us a place to add business rules later without changing the controllers.

```
public List<Article> findByCategory(String category) {
    if (category == null || category.trim().isEmpty()) return findAll();
    return articleRepository.findByCategoryIgnoreCase(category.trim());
}

public List<Article> search(String text) {
    if (text == null || text.trim().isEmpty()) return findAll();
    return articleRepository.findByTitleContainingIgnoreCase(text.trim());
}
```

Here the Service adds value: empty filter input falls back to listing all articles. Without this, the URL **/wiki?search=** would search for an empty string.

```
public Article save(Article article) {
    article.setLastUpdated(LocalDateTime.now());
    return articleRepository.save(article);
}
```

The most important Service method. Stamping the lastUpdated time is a business rule that lives here. The repository's save() method does an INSERT if article.id is null and an UPDATE if it has an id - JPA decides automatically.

```
public void deleteById(Long id) {
    articleRepository.deleteById(id);
}

public boolean existsById(Long id) {
```

```
    return articleRepository.existsById(id);
}
```

Delete and existence check. The controller uses existsById to decide whether to call deleteById or report 'not found'.

Likely viva question: “What does .save() do for new vs existing articles?”

HOW TO SAY IT: “It depends on whether the Article has an id. If id is null, JPA generates an INSERT statement and the database fills in the auto-generated id. If id is already set, JPA generates an UPDATE statement targeting the row with that id. That is why my updateArticle controller calls article.setId(id) before save - so JPA does the update.”

2.10 WikiController.java (Part B - Public Pages)

What is this file? The PUBLIC pages of the wiki - anyone can browse and read articles, no login needed.

Why does it exist? Just like a real wiki, anyone can read the content. Only logged-in admins can edit or delete.

```
@Controller
@RequestMapping("/wiki")
public class WikiController {
```

All URLs in this class start with /wiki. So /wiki shows the list, /wiki/3 shows article 3, /wiki?search=spring filters by search.

```
@GetMapping
public String listArticles(
        @RequestParam(value = "category", required = false) String category,
        @RequestParam(value = "search",   required = false) String search,
        Model model) {
```

Handles GET /wiki. **@RequestParam(required=false)** captures optional query string parameters. So /wiki?search=spring sets search='spring', /wiki on its own makes both null.

```
    List<Article> articles;
    if (search != null && !search.trim().isEmpty()) {
        articles = articleService.search(search);
    } else if (category != null && !category.trim().isEmpty()) {
        articles = articleService.findByCategory(category);
    } else {
        articles = articleService.findAll();
    }
```

Pick which Service method to call based on the URL parameters. If 'search' is provided, search by title. Otherwise if 'category' is provided, filter by category. Otherwise, list everything.

```
    model.addAttribute("articles", articles);
    model.addAttribute("category", category);
    model.addAttribute("search", search);
    return "wiki/list";
}
```

Put the list and the current filters into the Model so the template can show them. Render templates/wiki/list.html.

```
@GetMapping("/{id}")
public String viewArticle(@PathVariable("id") Long id, Model model) {

    Optional<Article> maybeArticle = articleService.findById(id);
    if (maybeArticle.isEmpty()) {
        model.addAttribute("errorMessage",
                "The article you requested does not exist.");
        return "error";
    }

    model.addAttribute("article", maybeArticle.get());
    return "wiki/view";
}
```

@PathVariable captures part of the URL path - so the URL /wiki/3 makes id = 3. We look up the article. If it does not exist (Optional is empty) we show the error page; otherwise we render wiki/view.html with the article.

Likely viva question: "What is the difference between @PathVariable and @RequestParam?"

HOW TO SAY IT: "@PathVariable captures part of the URL path itself - like the id in /wiki/3. @RequestParam captures a query string parameter - like search=spring in /wiki?search=spring. Path variables are part of the URL structure; query parameters are extra data tacked on to the end."

2.11 AdminController.java (Part B - CRUD)

What is this file? The admin-only pages: dashboard, create / edit / delete articles. This is where CRUD lives.

Why does it exist? The marker will likely focus most questions on this file because it covers the most rubric criteria: Controller, mappings, form handling, session-based auth, full CRUD.

```
@Controller
@RequestMapping("/admin")
public class AdminController {

    private final ArticleService articleService;

    @Autowired
    public AdminController(ArticleService articleService) {
        this.articleService = articleService;
    }
```

All URLs start with /admin. Dependency injection brings in the ArticleService.

```
private boolean isLoggedIn(HttpSession session) {
    return session.getAttribute(AuthController.SESSION_ADMIN_KEY) != null;
}
```

Helper method. Returns true if the session has an entry for our 'loggedInAdmin' key. We call this at the start of EVERY admin endpoint to check the user is logged in. If they are not, we redirect to /login.

```
@GetMapping
public String dashboard(HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";

    model.addAttribute("articles", articleService.findAll());
    model.addAttribute("adminUser",
            session.getAttribute(AuthController.SESSION_ADMIN_KEY));
    return "admin/dashboard";
}
```

READ: the dashboard. Loads all articles and passes them to the template along with the logged-in admin's name (so we can show 'Logged in as movindu' in the navbar).

```
@GetMapping("/new")
public String showCreateForm(HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
    model.addAttribute("article", new Article());
    model.addAttribute("mode", "create");
    return "admin/form";
}
```

CREATE step 1: show empty form. Put a blank Article into the model so the form has something to bind to. The 'mode' attribute tells the shared template whether to show 'New article' or 'Edit article' as the heading.

```
@PostMapping
public String createArticle(@Valid @ModelAttribute("article") Article article,
                            BindingResult bindingResult,
                            HttpSession session,
                            Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
```

```
    if (bindingResult.hasErrors()) {
        model.addAttribute("mode", "create");
        return "admin/form";
    }

    articleService.save(article);
    return "redirect:/admin";
}
```

CREATE step 2: handle submission. **@Valid** activates validation - the @NotBlank rules on Article are checked. **BindingResult** collects any errors. If validation failed, redisplay the form with errors. Otherwise save and redirect to the dashboard.

```
@GetMapping("/edit/{id}")
public String showEditForm(@PathVariable("id") Long id, ...) {
    if (!isLoggedIn(session)) return "redirect:/login";

    Optional<Article> maybeArticle = articleService.findById(id);
    if (maybeArticle.isEmpty()) { ... return "error"; }

    model.addAttribute("article", maybeArticle.get());
    model.addAttribute("mode", "edit");
    return "admin/form";
}
```

UPDATE step 1: show form pre-filled with the existing article. Same template (admin/form.html), but with mode='edit' and the article fully populated.

```
@PostMapping("/edit/{id}")
public String updateArticle(@PathVariable("id") Long id,
                            @Valid @ModelAttribute("article") Article article,
                            BindingResult bindingResult, ...) {
    ...
    article.setId(id);   // <-- KEY LINE
    articleService.save(article);
    return "redirect:/admin";
}
```

UPDATE step 2: save changes. The crucial line is **article.setId(id)** - without it, JPA would treat this as a new article and INSERT a new row instead of UPDATING the existing one. Setting the id tells JPA: 'this is the article with id X; update that row'.

```
@GetMapping("/delete/{id}")
public String confirmDelete(...) {
    ...
    return "admin/confirm-delete";
}

@PostMapping("/delete/{id}")
public String deleteArticle(@PathVariable("id") Long id, HttpSession session) {
    if (!isLoggedIn(session)) return "redirect:/login";
    if (articleService.existsById(id)) articleService.deleteById(id);
    return "redirect:/admin";
}
```

DELETE in two steps. The GET shows a confirmation page so the user can cancel. Only the POST actually deletes. Using POST instead of GET for delete is important: GET requests should never change data, because browsers prefetch them, web crawlers follow them, and they can be cached. POST is fired only by an explicit form submission.

Likely viva question: “Why is delete a two-step process with a confirmation page?”

HOW TO SAY IT: “Two reasons. First, accidental deletion is destructive and cannot be undone, so the user should confirm. Second, GET requests must never change data - browsers prefetch them, crawlers follow them, they can be cached. By using a confirmation page (GET) followed by an actual delete (POST), the destructive action only fires when the user explicitly submits the form.”

2.12 application.properties

What is this file? Configuration file. Spring Boot reads it at startup.

Why does it exist? Properties files let us change behaviour without touching code. Port number, database URL, etc. all live here.

```
server.port=8080
```

Tomcat listens on port 8080. If something else is already using that port, change to 8081 or similar.

```
spring.thymeleaf.cache=false
```

Disable template caching during development so HTML edits show up immediately.

```
spring.datasource.url=jdbc:h2:file:./data/wikidb;AUTO_SERVER=TRUE
spring.datasource.username=sa
spring.datasource.password=
```

Connect to an H2 database. **file:./data/wikidb** means data is stored in a file on disk inside the data folder of our project. **AUTO_SERVER=TRUE** lets multiple processes (the app + the H2 console) read it at the same time.

```
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Enables a built-in web page at **/h2-console** where we can view the database tables. Useful during the demo to show the data is really being stored.

```
spring.jpa.hibernate.ddl-auto=update
```

update means: at startup, compare our @Entity classes to the existing tables. Create missing tables, add missing columns. Never drop or modify existing data.

```
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
```

Run our data.sql script AFTER Hibernate has built the tables (otherwise the inserts would fail because the tables would not exist yet).

Likely viva question: “What would happen if I deleted the data folder?”

HOW TO SAY IT: “The H2 database files would be gone, so on next startup Hibernate would create empty tables. data.sql would re-seed the admin and the five sample articles. Any custom articles I created during testing would be lost - but the sample data would come back automatically.”

2.13 data.sql (Part B)

What is this file? SQL script that seeds the database with starter data.

Why does it exist? Without this, the database would be empty on first run. We seed one admin account and five sample articles.

```
MERGE INTO ADMIN (id, username, password) KEY(id)
    VALUES (1, 'movindu', '123');
```

MERGE means 'insert if a row with this id does not exist; otherwise update it'. So we can run this script repeatedly - it will not crash on duplicate keys. The seeded admin uses username 'movindu' and password '123' - matching the Part A specification.

```
MERGE INTO ARTICLE (id, title, category, content, last_updated) KEY(id) VALUES
    (1, 'Welcome to WikiApp', 'General', '...', CURRENT_TIMESTAMP);
-- ... and four more sample articles ...
```

Five sample articles so the public wiki has something to display on first run. Each gets a fixed id (1-5) and the current timestamp.

```
ALTER TABLE ARTICLE ALTER COLUMN ID RESTART WITH 100;
ALTER TABLE ADMIN   ALTER COLUMN ID RESTART WITH 100;
```

This line fixes a real bug we encountered. When we seed rows with ids 1-5, the database's auto-increment counter still starts at 1. So when we create the next article, JPA picks id=1 and we get a primary-key collision. RESTART WITH 100 bumps the counter so newly created rows get id 100, 101, 102 - safely past the seeded ones.

Likely viva question: “Why are you using MERGE instead of INSERT?”

HOW TO SAY IT: “MERGE will INSERT if no row with the given id exists, but UPDATE the existing row if one does. INSERT would crash on the second run because the rows would already exist. MERGE makes the script idempotent - safe to run repeatedly.”

2.14 Templates (HTML files)

Each template is a normal HTML file with extra th:* attributes. The templates are simple - the heavy lifting is done by the Java code.

login.html

The login form. Binds to a LoginForm via th:object. The form's submit URL is /login and the method is POST. Two text inputs (one type=password) and two buttons.

welcome.html / error.html / register.html

Simple display pages. error.html is the most useful - it is reused for both 'login failed' and 'article not found' situations.

wiki/list.html

The PUBLIC home page of the wiki. Has a search box at the top and a grid of article cards. Uses **th:each** to loop over the articles list. Uses **#strings.abbreviate(content, 140)** to show a 140-char preview.

wiki/view.html

Single-article page. Shows the title, category badge, last-updated time and full content. Uses **th:text** which auto-escapes HTML, so even if an admin somehow puts <script> tags in an article, they appear as plain text - never executed.

admin/dashboard.html

The admin home page. Shows every article in a table with Edit and Delete buttons. The top bar greets the admin by name (read from the session attribute the controller put into the model).

admin/form.html

ONE template used for both creating and editing. The 'mode' attribute decides the heading and the form-action URL. Uses **th:errors** and **#fields.hasErrors()** to show validation messages under each field.

admin/confirm-delete.html

'Are you sure?' page. The actual delete is a POST form (not just a link) so deleting requires an explicit click and cannot be triggered by URL prefetching, browser caching or accidental clicks.

PART 3

The demo script

3.1 Before the viva starts

Have these three things open in this order:

1. A terminal in the WikiApp folder with the app already running. Start it with **mvn spring-boot:run** at least 5 minutes before so it is fully warmed up.

2. A browser window already on http://localhost:8080/wiki - so the marker can see the public page first.

3. Your code editor (VS Code or IntelliJ) showing the project structure in the side panel.

WATCH OUT: Check that your network is not in 'mobile hotspot' mode and that no VPN is interfering with localhost. Test http://localhost:8080 in the browser before the marker arrives.

3.2 The opening line

HOW TO SAY IT: “Hi, I built WikiApp for BIT235 in two parts. Part A is a Spring Boot login screen using the MVC pattern with three layers: Controller, Service and Repository. Part B added a real H2 database, a public wiki for browsing articles, and an admin back-end with full CRUD - create, read, update and delete. Let me walk you through it.”

3.3 The demo flow (in order)

Step 1 - Show the public wiki

Go to http://localhost:8080/wiki. Five sample articles appear in cards.

HOW TO SAY IT: “This is the public wiki. Anyone can browse without logging in. The articles are loaded from the H2 database I configured in Part B.”

• Click on 'What is MVC?' to show the single-article view.

• Click 'Back to all articles', then type 'spring' in the search box and click Search.

HOW TO SAY IT: “The search uses a custom query method I declared in ArticleRepository. Spring Data automatically generated the SQL from the method name.”

Step 2 - Try logging in with wrong credentials

Click 'Admin login'. Enter wrong password. Show the error page.

HOW TO SAY IT: “If the credentials are wrong, the controller adds an error message to the model and returns the error view name.”

Step 3 - Log in correctly

Username: **movindu** Password: **123**

You land on the admin dashboard.

HOW TO SAY IT: “On successful login, the controller stored my username in the HttpSession and redirected to /admin. The session lets the admin pages know I am logged in across multiple requests.”

Step 4 - Create an article

Click 'New article'. Fill in title, category and content. Submit.

HOW TO SAY IT: “That POST to /admin went to AdminController.createArticle. @ModelAttribute built an Article object from the form fields, @Valid checked the @NotBlank rules, and ArticleService.save() inserted a new row in the database with an auto-generated id.”

Step 5 - Edit the article

Click Edit on your new article. Change the title. Save.

HOW TO SAY IT: “Same template as create - the 'mode' attribute decides whether the form posts to /admin or /admin/edit/{id}. Setting article.setId(id) before save() makes JPA do an UPDATE rather than an INSERT.”

Step 6 - Delete the article

Click Delete. Show the confirmation page. Click 'Yes, delete'.

HOW TO SAY IT: “Two-step delete - GET shows the confirmation, only the POST actually deletes. I used POST because GET requests should never change data; browsers prefetch them and crawlers follow them.”

Step 7 - Log out and try /admin again

Click Log out. The browser goes back to /login. Then manually type http://localhost:8080/admin in the address bar. You get bounced back to the login page.

HOW TO SAY IT: “Logout calls session.invalidate() which throws away the stored username. The isLoggedIn helper now returns false, so every admin endpoint redirects to /login.”

Step 8 - Optional: show the H2 console

Open http://localhost:8080/h2-console. JDBC URL: **jdbc:h2:file:./data/wikidb**. User: **sa**. Password: empty. Click Connect. Run **SELECT * FROM ARTICLE;**.

HOW TO SAY IT: “This is the actual H2 database showing the articles I just created. Hibernate generated the schema from my @Entity classes.”

PART 4

Likely viva questions & answers

Read each question, try to answer out loud BEFORE looking at the answer. Then compare. The answers below are what you should aim for - your version can be slightly different as long as the key ideas are there.

Q: Tell me about your project in one paragraph.

A: WikiApp is a Spring Boot project I built in two parts. Part A was a simple login screen using MVC with three layers - Controller, Service and Repository. Part B replaced the hard-coded credentials with a real H2 database and added a public wiki for browsing articles plus an admin back-end with full CRUD. Login uses HttpSession to remember the logged-in admin across requests.

Q: Why three layers and not just one?

A: Separation of concerns. Controllers handle HTTP, Services hold business rules, Repositories talk to the database. When I added the database in Part B, only the Repository layer changed - the Controller and Service did not need to be rewritten. That is exactly the benefit of layering.

Q: Where is your business logic?

A: In the Service classes - AuthService and ArticleService. The validation rules, the trim and lower-case behaviour, the time-stamping on save - all of that is in Services. Controllers only route, Repositories only fetch.

Q: What is the difference between @GetMapping and @PostMapping?

A: @GetMapping handles HTTP GET requests, which are used when the user just wants to view a page and should not change anything on the server. @PostMapping handles POST requests, which are form submissions or other data-changing actions.

Q: Why does delete use POST instead of GET?

A: GET requests must be safe - they are prefetched by browsers, followed by web crawlers, and can be cached. If delete were a GET, any of those could accidentally wipe data. POST only fires on explicit form submission.

Q: What does redirect: do?

A: It tells Spring to send a 302 HTTP response with a Location header pointing elsewhere, instead of rendering a template. The browser then makes a fresh GET request to that location. We use it after login and after every CRUD action so the user can refresh the page without resubmitting the form.

Q: How does Spring know to put AuthService into the Controller?

A: AuthService is marked @Service so Spring creates one instance at startup. The Controller's constructor declares it needs an AuthService and is marked @Autowired, so Spring passes the same instance into the constructor. This is constructor dependency injection.

Q: Why use constructor injection instead of field injection?

A: Two reasons. The dependencies are explicit - just by reading the constructor I see exactly what the class needs. And the fields can be marked final, so they cannot be reassigned after construction - safer code.

Q: What is a bean?

A: An object that Spring manages on my behalf. I do not create it with new - Spring creates it once at startup, keeps a reference, and gives it to anyone that asks via constructor injection.

Q: How does data typed in the form end up in your Java code?

A: The HTML inputs are bound via th:field. When the form is submitted, Spring sees @ModelAttribute LoginForm in my controller, creates a new LoginForm object, calls setUsername and setPassword with the typed values, then passes that filled-in object into my method.

Q: What is Model?

A: It is a container Spring provides for data going to the view. I put values in with addAttribute. Thymeleaf reads from it when rendering the page, so ${username} in the template prints whatever I stored under the name 'username'.

Q: What is the difference between ${...} and *{...} in Thymeleaf?

A: ${variable} reads from the Model directly. *{property} is a shortcut that reads from the object the form is bound to via th:object. Inside a form, *{username} is shorthand for ${loginForm.username}.

Q: How does JPA know what tables to create?

A: Hibernate scans for @Entity classes at startup. For each one it makes sure a table with the same name exists, with columns matching the fields. With ddl-auto=update it creates missing tables and adds new columns automatically, but never drops data.

Q: What is JpaRepository?

A: A Spring Data interface that provides standard CRUD methods - findAll, findById, save, deleteById and many more. By extending it my interfaces inherit all those methods. Spring writes the implementation class at runtime, including all the SQL.

Q: How does findByCategoryIgnoreCase work?

A: Spring Data parses the method name. findBy means SELECT, Category matches the field name, IgnoreCase wraps both sides in LOWER. So the generated SQL is SELECT * FROM article WHERE LOWER(category) = LOWER(?). I never wrote that SQL - I just declared the method signature.

Q: Why Optional<Admin> instead of just Admin?

A: Because the lookup might find nothing. With plain Admin the method would have to return null, and the caller could forget to check, leading to a NullPointerException. Optional is a wrapper that forces the caller to handle the missing case explicitly.

Q: What does .save() do for new vs existing entities?

A: If the entity's id is null, JPA does an INSERT and the database fills in a new id. If the id is already set, JPA does an UPDATE on the row with that id. That is why my updateArticle controller does article.setId(id) before save - so JPA updates instead of inserting.

Q: How is the admin area protected?

A: Every method in AdminController calls isLoggedIn(session) first. That helper checks whether session.getAttribute('loggedInAdmin') is non-null. If the user is not logged in, the method returns 'redirect:/login'.

Q: How does HttpSession actually work?

A: When a browser first hits the site, Tomcat assigns it a long random session id and sends it back as a cookie. The browser sends that cookie on every later request, so Tomcat knows which session to

load. The session is just an in-memory key-value store tied to that id.

Q: Why are you not using Spring Security?

A: Two reasons. The brief did not require it. And manual session checks are easier to explain in the viva - I can show every line that does authorisation. In a real production app I would absolutely use Spring Security.

Q: What is @Valid?

A: It tells Spring to run the validation rules - like @NotBlank - on the bound object before my method runs. If anything fails, the errors are collected in a BindingResult I can inspect.

Q: What is @NotBlank?

A: A validation rule from Bean Validation. It says the field must not be null and must not be empty or whitespace-only. If the user submits an empty title, the form is rejected and the error message I supplied is shown.

Q: What is encapsulation?

A: An OOP principle: hide the internal state of an object behind methods. My LoginForm has private fields and exposes them only through getters and setters. Other classes cannot access the fields directly.

Q: What is a POJO?

A: Plain Old Java Object - a simple class with private fields, a no-args constructor, and getters and setters. LoginForm is a POJO. Article and Admin are POJOs too, just with extra annotations to make them entities.

Q: Why does the Article entity need a no-args constructor?

A: JPA requires it. When loading rows from the database, Hibernate calls the no-args constructor to create a blank Article and then uses reflection to fill in the fields. Without the no-args constructor, Hibernate could not build the object.

Q: Could I have put validation in the Controller instead of the Service?

A: Technically yes, but it would mix concerns. The Controller would have routing code AND business rules. By keeping validation in the Service, the rules are reusable, the Controller stays small, and I can write unit tests for the validation without involving HTTP at all.

Q: If the database is empty, where does the seed data come from?

A: data.sql in src/main/resources/. Spring Boot runs it automatically at startup - but only AFTER Hibernate creates the tables, thanks to spring.jpa.defer-datasource-initialization=true. The MERGE statements make it safe to run repeatedly.

Q: What if two users try to register with the same username?

A: The database itself rejects it because I marked the username column @Column(unique = true). The save() call would throw a DataIntegrityViolation exception. We did not implement self-registration, but if we did, that error would surface to the user as 'username already taken'.

Q: What would you change first if you had more time?

A: Hash passwords with BCrypt instead of storing them plain text - that is the single biggest real-world weakness. Then maybe add Spring Security for proper auth, and pagination on the article list.

Q: (If you genuinely do not know an answer)

A: I am not 100% sure, but my best guess is X because Y. Let me check my code to confirm. If the marker pushes, say: I would need to look that up to be sure.

PART 5

Quick-reference cheat sheet

Cheat Sheet

Glance at this right before the viva.

The architecture in one sentence

Controller routes, Service decides, Repository fetches.

The login flow in one paragraph

Browser POSTs username and password to /login. AuthController calls AuthService.isAuthenticated which trims and lower-cases the username, calls AdminRepository.findByUsername to fetch the admin from H2, and compares the passwords with .equals. If valid, the controller stores the username in HttpSession and redirects to /admin.

The CRUD flow in one paragraph

AdminController has paired GET/POST endpoints for create and edit, plus a two-step delete. Each endpoint first checks the session, then calls ArticleService.save() or deleteById(). save() does an INSERT if the id is null and an UPDATE if it has an id - JPA decides automatically.

Most likely viva questions (top 6)

1. Walk me through what happens when I click Log in.
2. Why three layers? What does each layer do?
3. How does the username typed in the form end up in your Java code?
4. What is JpaRepository and who writes the SQL?
5. How is the admin area protected?
6. Why is delete a two-step process?

Annotations cheat sheet

@SpringBootApplication	Bundle of @Configuration + @EnableAutoConfiguration + @ComponentScan.
@Controller	Web controller. Methods return view names.
@Service	Business-logic bean.
@Repository	Data access bean.
@Autowired	Inject the dependency. Used on constructors.
@RequestMapping("/x")	URL prefix at class level.
@GetMapping	Handle HTTP GET.
@PostMapping	Handle HTTP POST.
@ModelAttribute	Build a Java object from form fields.

@PathVariable	Capture a value from the URL path.
@RequestParam	Capture a query string parameter.
@Valid	Run validation on the bound object.
@NotBlank	Field must not be null/empty/whitespace.
@Entity	Maps this class to a database table.
@Id	Primary key field.
@GeneratedValue	Database generates the value.
@Column	Configure the database column.

If something breaks during the demo

Port 8080 already in use	Change server.port in application.properties to 8081.
404 on /wiki	Did you visit /wiki not / ? Spring needs the /wiki prefix.
Login does not work	Username is movindu, password is 123. Both case-sensitive (well, username is auto-lowercased).
H2 console will not connect	JDBC URL: jdbc:h2:file:./data/wikidb. User: sa. Password: empty.
App will not start	Stop any old Java processes (Get-Process java | Stop-Process -Force in PowerShell).
You forget an answer	Open the file in question and read the comments out loud. The comments cover every annotation.

The one thing the marker most wants to hear

HOW TO SAY IT: “I separated my code into three layers - Controller, Service, Repository - so each class has one job. When I added the database in Part B, only the Repository changed. That is the benefit of layered design.”

End of guide - good luck with the viva!